COMP0215 Coursework: Reinforcement Learning in Simple Games

Technical Report

Ashraya Poudel Mehul Chourasia Arzaan Kapadia Aayan Islam

March 29, 2026

1 Introduction

This report describes our approach to developing a self-learning reinforcement learning agent for Gomoku (Five-in-a-Row) on a 9×9 board (with an extension to 15×15). The objective is to place five stones in an unbroken row, column, or diagonal before the opponent. Despite its simple rules, Gomoku presents a considerable challenge for search-based methods: on a 15×15 board there are up to 225 legal moves at each step, and deep tactical combinations demand both local pattern recognition and long-range planning.

Our approach is based on DeepMind's AlphaZero framework [Silver et al., 2017], which combines Monte Carlo Tree Search (MCTS) with a deep residual neural network that simultaneously predicts move probabilities and game outcomes. Crucially, AlphaZero requires no domain-specific handcrafted features beyond the rules of the game; all strategic knowledge is learned from scratch through iterated self-play.

Rather than committing to a single fixed configuration, we treat this as a **comparative study** of the AlphaZero framework applied to Gomoku. Two independent implementations were developed in parallel within the team, exploring different training paradigms:

1. **Pure self-play**: A larger network ($\sim$2M parameters) trained exclusively through self-play, with batched GPU inference and 400 MCTS simulations per move. This approach is closest to the original AlphaZero paper.
2. **Mixed-opponent training**: A smaller, faster network ($\sim$200k parameters) trained on a mixture of self-play (70%) and games against a rule-based heuristic opponent (30%), with 200 MCTS simulations per move. This approach bootstraps tactical awareness through structured training signal.

Both pipelines share the same core architecture (residual policy-value network + MCTS with PUCT selection), allowing us to isolate the effects of network capacity, training signal source, and hyperparameter choices. The pure self-play pipeline was further iterated through five major versions (v1–v5), each addressing limitations identified in prior runs, providing an ablation study of individual design decisions.

1.1 Why AlphaZero for Gomoku?

Several properties of Gomoku make it well-suited to this paradigm:

- **Strategic depth**: The game requires both local pattern recognition (detecting threats and opportunities within a small neighbourhood of stones) and long-range planning (building multi-directional traps). Residual convolutional networks are naturally suited to learning such hierarchical spatial structure.

- **Perfect information**: Both players observe the full board state at all times, so there is no hidden state to model.
- **Two-player zero-sum structure**: Every win for one player is an equal loss for the other, meaning a single value head can express both players' perspectives via sign-flip — no separate value functions are needed.
- **Rich symmetry**: The board is invariant under the dihedral group D_4 (four rotations and two reflections), giving eight equivalent board representations per position. This can be exploited during training to multiply data efficiency by a factor of eight at negligible cost.

An important alternative we considered was Q-learning, which had been explored by some of our team (in particular Double Deep Q-learning). However, these models performed significantly worse due to the reactive, pattern-memorisation nature of Q-learning, which struggles with Gomoku's sparse rewards (the outcome is only revealed at the end of a potentially 81-move game). AlphaZero's combination of neural network intuition with explicit MCTS lookahead allows it to actively plan and calculate winning sequences rather than relying on memorised state-action mappings.

2 Methodology

This section describes the shared components of our AlphaZero framework. Both training pipelines (pure self-play and mixed-opponent) use the same core architecture and MCTS algorithm; the pipeline-specific differences in network scale, optimiser, and training signal are detailed in Section 5.

2.1 Overview: AlphaZero-Style Training

Both pipelines iterate three phases in a loop:

1. **Data generation**: The current network plays games (against itself and, in the mixed-opponent pipeline, against a heuristic opponent), using MCTS to select moves. Each game produces training tuples of `(board_state, mcts_policy, game_outcome)`.
2. **Network training**: The network is updated by gradient descent on the collected data, supervised by the MCTS policies and game outcomes as targets.
3. **Evaluation** (mixed-opponent pipeline): Periodically, the updated network is tested against a random agent and the heuristic opponent to track progress objectively.

Over many iterations the network improves, producing stronger self-play partners, which in turn generate richer training data — a virtuous cycle of improvement.

2.2 State Representation

The game state is encoded as a *perspective-normalised* tensor of shape $3 \times \text{size} \times \text{size}$:

- **Plane 0**: Current player's stones (1 where occupied, 0 otherwise).
- **Plane 1**: Opponent's stones (1 where occupied, 0 otherwise).
- **Plane 2**: Differs between pipelines — this is discussed in Section 5. The pure self-play pipeline encodes the current player's colour (all-ones for Black, all-zeros for White), while the mixed-opponent pipeline uses a constant all-ones bias plane.

By always placing the current player's stones in plane 0 regardless of colour, the same network weights generalise across both Black and White play, halving the effective state space the network must learn.

2.3 Monte Carlo Tree Search

MCTS is used both during data generation and during inference. Both pipelines use the **PUCT** (Polynomial Upper Confidence Trees) selection criterion, following the AlphaZero paper:

$$\text{score}(s,a) = -Q(s,a) + c_{\text{puct}} \cdot P(s,a) \cdot \frac{\sqrt{N(s)}}{1+N(s,a)}$$

where:

- $Q(s,a)$ is the mean action value estimated from backed-up simulations (negated to account for alternating turns)
- $P(s,a)$ is the prior probability from the neural network's policy head
- $N(s)$ is the visit count of the parent node
- $N(s,a)$ is the visit count of the child
- $c_{\text{puct}} = 1.5$ is the exploration constant, balancing exploitation of high-Q moves with exploration of high-prior moves

Each simulation traverses the tree from the root, expanding a new leaf node and evaluating it with the neural network (which replaces the traditional random rollout), then backs up the value in negamax fashion. After a fixed number of simulations, the move is sampled from the resulting visit count distribution. The number of simulations per move varies between pipelines (200–400 during training; see Table 6).

Tree reuse: Both implementations retain the subtree rooted at the played move after each turn, carrying forward all accumulated visit counts and value estimates. This amortises computation over successive moves and significantly deepens the effective search without additional simulation budget.

Exploration noise: To ensure the agent explores novel lines during self-play (and does not simply reinforce a fixed dominant strategy), Dirichlet noise is mixed into the root's prior distribution:

$$P_{\text{root}}(a) = (1-\varepsilon) \cdot P_{\text{net}}(a) + \varepsilon \cdot \eta_a, \quad \eta \sim \text{Dir}(\alpha)$$

with $\alpha = 0.3$ in both pipelines (the noise mixing weight ε is 0.35 in the pure self-play pipeline and 0.25 in the mixed-opponent pipeline).

Temperature annealing: For the first several moves of each game, moves are sampled proportionally to visit counts (high temperature, encouraging opening diversity). After a threshold (8–10 moves, depending on pipeline), the agent selects the most-visited move deterministically (greedy exploitation).

2.4 Neural Network Architecture

Both pipelines use the same dual-headed residual architecture. A **stem** block applies a 3×3 convolution (with batch normalisation and ReLU) to project the 3-channel input into the working channel width C. This is followed by a **residual tower** of N blocks, each of the form:

```
x -> Conv(C->C, 3x3) -> BN -> ReLU -> Conv(C->C, 3x3) -> BN -> (+skip) -> ReLU
```

Residual connections allow gradients to flow without vanishing through many layers, enabling the network to learn both low-level local patterns (two- or three-in-a-row threats) and higher-level strategic structure simultaneously.

The network capacity was a key experimental variable. Table 1 summarises the configurations tested across both pipelines.

Table 1: Network configurations tested across experiments.

Configuration	**Res. Blocks**	**Filters (C)**	**Value FC**	**Params**
Small (pure self-play v1–v3, mixed-opponent)	4–5	64	64	∼200–390k
Medium (pure self-play v4)	7	96	64	∼900k
Large (pure self-play v5)	10	128	256	∼2.0M
15×15 extension	6	128	64	∼1.0M

Policy head

A 1×1 convolution reduces the feature maps to 2 channels, which are then flattened and passed through a linear layer projecting to size^2 logits. Illegal moves (occupied positions) are masked to $-\infty$ before applying softmax, ensuring the network never assigns probability to invalid moves.

Value head

A 1×1 convolution reduces to 1 channel, followed by flattening and two linear layers (size$^2 \to H \to 1$, where H is the hidden size from Table 1) with a ReLU in between. A final tanh activation bounds the output to $[-1, +1]$, where $+1$ means the current player is predicted to win with certainty and -1 means certain loss.

2.5 Training Procedure

The training procedure is shared across both pipelines: a continuous cycle of generating games, computing the loss, and optimising the network.

Loss function: The network is trained using a combined loss:

$$\mathcal{L} = \mathcal{L}_{\text{policy}} + \lambda_v \cdot \mathcal{L}_{\text{value}} + c\|\theta\|^2$$

$$\mathcal{L}_{\text{policy}} = -\sum_a \pi^{\text{MCTS}}(a) \log \pi_\theta(a)$$

$$\mathcal{L}_{\text{value}} = (v_\theta - z)^2$$

where:

- The **policy loss** $\mathcal{L}_{\text{policy}}$ is the cross-entropy between the MCTS visit-count distribution π^{MCTS} and the network's predicted policy π_θ. The network learns to intuitively predict the moves that MCTS calculates through explicit search.
- The **value loss** $\mathcal{L}_{\text{value}}$ is the mean squared error between the network's evaluation v_θ and the true game outcome $z \in \{-1, 0, +1\}$.
- The **value loss weight** λ_v controls the relative importance of the two heads. We experimented with $\lambda_v \in \{0.5, 1.0, 2.0\}$ across pipelines (see Section 5).
- $c = 10^{-4}$ is the L2 regularisation coefficient, preventing the network from memorising specific board states.

Optimiser: Both pipelines use momentum-based optimisation with cosine annealing of the learning rate. The pure self-play pipeline uses SGD with momentum 0.9 (initial LR 0.01 $\rightarrow$ 5×10^{-4}), while the mixed-opponent pipeline uses Adam (initial LR 0.002 $\rightarrow$ 1×10^{-4}). The cosine schedule provides smooth, sustained decay appropriate for the continuously shifting data distribution of self-play.

Replay buffer: Both pipelines store training tuples $(s, \boldsymbol{\pi}, z)$ in a circular replay buffer holding up to 200,000 examples. During training, random mini-batches are drawn from this buffer (batch size 256 or 512, depending on pipeline). Shuffling prevents the network from overfitting to the sequential structure of individual games. As new data is generated, the oldest examples are replaced, ensuring the agent trains on its most recent play.

Data augmentation: The 9×9 Gomoku board has the dihedral symmetry group D_4 (four rotations $\times$ two reflections $= 8$ transformations). Both pipelines apply all eight symmetries to every training position, effectively multiplying the training data by $8\times$ at no additional self-play cost. This dramatically accelerates learning by exposing the network to each tactical pattern in all orientations.

Training loop: Each iteration follows these steps:

1. **Play**: Generate 50 games using MCTS (pure self-play generates all games against itself; mixed-opponent plays 30% of games against the heuristic opponent).
2. **Store**: Apply dihedral symmetries to the resulting data and add it to the replay buffer.
3. **Learn**: Perform 200–300 gradient steps on the network using random mini-batches from the buffer.
4. **Evaluate** (mixed-opponent pipeline only): Every 5 iterations, test win rate against the heuristic and random opponents. Save a checkpoint if performance improves.

3 Heuristic Opponent

To objectively measure how smart our AI is getting, we need a baseline to test it against. For this, we built a rule-based heuristic opponent. This opponent does not use machine learning; it simply follows a strict checklist of logical rules to decide its next move:

1. **Immediate win**: First, it scans the board. If it can place a stone to create an immediate five-in-a-row, it plays there instantly.
2. **Immediate block**: If it cannot win immediately, it checks if the other player is about to win. If the opponent has four-in-a-row, the heuristic will immediately play the blocking move.
3. **Scored candidate moves**: If neither an immediate win nor an immediate block is required, the heuristic looks at all empty squares within a two-square radius of any existing stones. It gives each square a score based on two factors:
   - **Attack score**: How many of its own lines (of 2, 3, or 4 stones) would intersect at this square?
   - **Defence score**: How many of the opponent's lines would be interrupted by placing a stone here?
   - **Combined**: It combines these scores using the formula $1.1 \times \text{attack} + \text{defence}$, giving a slight preference to aggressive, offensive play.

   The bot then plays on the square with the highest total score.

Consistently beating this heuristic opponent is a major milestone for our RL agent. To defeat it, the neural network cannot just play randomly; it must learn to plan ahead, set up multi-directional traps (like "forks"), and outmanoeuvre an opponent that will never miss an obvious block.

4 Experiments and Quantitative Analysis

Our development followed an iterative process through five major training script versions (v1–v5), each addressing limitations identified in prior runs. This section presents the hyperparameter landscape, training dynamics, and ablation findings.

4.1 Iterative Development and Version Summary

Table 2 summarises the cumulative changes across versions.

Table 2: Summary of training script versions and their cumulative changes.

Version	**Key Changes**
v1 (`alpha_zero1`)	Baseline: 5 blocks / 64 filters, 200 sims, no augmentation, no tree reuse, no LR scheduling.
v2 (`alpha_zero2_chk`)	Fixed MCTS backup sign convention. Added persistent tree reuse across moves. Added dihedral data augmentation (8× training data). Added multi-step LR decay. First version trained end-to-end: 47 iterations, then extended to 77.
v3 (`az_gomuku3`)	Batched self-play for GPU efficiency. Curriculum MCTS (100 → 200 → 400 sims). Gradient clipping (max norm 1.0). Gentler LR decay ($\gamma = 0.5$).
v4 (`az_gomuku4`)	Scaled network to 7 blocks / 96 filters. Higher initial LR (0.02). Lowered Dirichlet α to 0.15. Value loss weight $\lambda_v = 0.5$. Cosine annealing LR. Buffer flush on curriculum transitions.
v5 (`az_gomuku5`)	Scaled network to 10 blocks / 128 filters (∼2M params). Value head FC widened to 256. Buffer 200k. Fixed 400 sims, 300 iterations. Cosine annealing ($\eta_{\min} = 5 \times 10^{-4}$). Returned to proven recipe from v2 + scale-up.

The overall narrative is: *baseline works → too many simultaneous changes breaks it → return to basics reveals capacity bottleneck → principled scale-up succeeds.*

4.2 Hyperparameter Tuning

Table 3 consolidates the hyperparameter settings across all experiments.

Key observations from hyperparameter tuning:

- **LR decay rate**: The v2 baseline used aggressive multi-step decay ($\gamma = 0.1$), which halved the LR to $\sim 10^{-4}$ by milestone 50, then $\sim 10^{-5}$ by milestone 75. This effectively killed all learning after iteration 47, explaining the stagnation between iterations 47–77. The switch to $\gamma = 0.5$ (v3) and ultimately cosine annealing (v4/v5) provided sustained, smooth decay appropriate for the continuously shifting data distribution of self-play.
- **Simultaneous changes**: Versions 3 and 4 each changed multiple hyperparameters at once, making it impossible to isolate which change helped or hurt. Version 3 combined curriculum MCTS (starting with only 100 sims — producing very noisy early targets),

Table 3: Hyperparameter comparison across training runs (v2–v5).

Parameter	**v1**	**v2 (baseline)**	**v3**	**v4**	**v5**
Residual blocks	5	5	5	7	10
Filters	64	64	64	96	128
Approx. parameters	∼390k	∼390k	∼390k	∼900k	∼2.0M
MCTS sims	200	200	100→400	100→400	400
Dirichlet α	0.3	0.3	0.3	0.15	0.3
Buffer size	50k	50k	50k	100k	200k
Initial LR	0.01	0.01	0.01	0.02	0.01
LR schedule	None	Multi-step (γ=0.1)	Multi-step (γ=0.5)	Cosine	Cosine
Gradient clipping	No	No	Yes (1.0)	Yes (1.0)	No
Value loss weight λ_v	1.0	1.0	1.0	0.5	1.0
Data augmentation	No	Yes (8×)	Yes (8×)	Yes (8×)	Yes (8×)
Tree reuse	No	Yes	Yes	Yes	Yes
Total iterations	–	77	79	100	300
Outcome	Never trained	Learned blocking; plateau at iter 47–77	Worse than v2	Still not good	**Strongest agent**

gradient clipping (possibly too restrictive at 1.0 for the small network), and a new LR scheduler. Version 4 further compounded changes: a larger network, higher initial LR (0.02), lower Dirichlet α, buffer flushing on curriculum transitions, and more. Both underperformed the v2 baseline. This served as a cautionary lesson: *change one variable at a time.*

- **Buffer flush**: In v4, the replay buffer was emptied at each curriculum transition to avoid stale data from lower sim-count games contaminating training. In practice, this forced the network to relearn from scratch mid-training while simultaneously adjusting LR, sim count, and training steps — a recipe for instability.

4.3 Learning Curves

4.3.1 Training run V5

Training loss data from v5 (our best model) reveals clear phases. Table 4 shows the loss trajectory and its decomposition into policy and value components.

Table 4: v5 training loss trajectory over selected iterations.

Iteration	**Total Loss**	**Policy Loss**	**Value Loss**	**LR**
119	2.4479	–	–	0.006765
130	2.4441	–	–	0.006238
140	2.4478	–	–	0.005747
144	2.4603	2.3613	0.0990	–
150	2.4277	2.3372	0.0905	0.005250

Several observations emerge:

- The total loss plateaued at ∼2.43–2.46 for over 30 consecutive iterations (iters 119–150), indicating the model had stopped improving meaningfully.
- The value loss was already very small (∼0.09), meaning the value head had effectively converged — the network correctly predicted game outcomes.
- The policy loss dominated (∼2.36), corresponding to the network spreading probability across approximately 10 plausible moves per position. This is consistent with Gomoku having multiple reasonable candidate moves in most board states.
- The 200k replay buffer was fully saturated, replacing old data 1:1 with new data of essentially the same quality.

A head-to-head evaluation between the iteration-118 and iteration-143 checkpoints confirmed the plateau: over 100 games (50 per colour), the result was 18 wins, 62 draws, and 20 losses. The two models were the same strength despite 25 additional iterations of training. The 62% draw rate is itself informative — it shows both models had learned strong defensive play but could not break through offensively.

4.3.2 Mixed-opponent training (155 iterations)

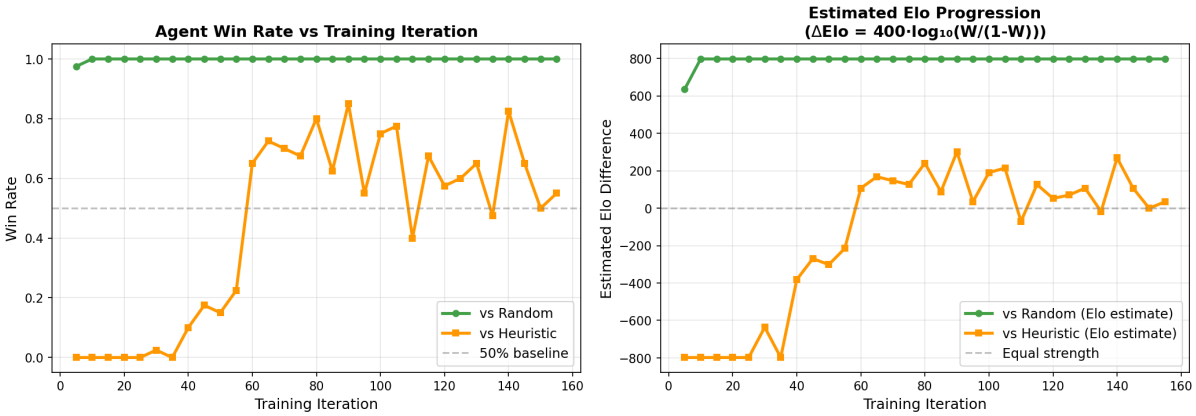

Figure 1: Winrate over training iterations

The agent achieved near-perfect win rates against the random opponent within the first 10-15 iterations and maintained this throughout training, corresponding to an estimated Elo advantage of over +800. Performance against the smarter heuristic opponent, started at 0 percent for most of the beggining and spiked around iteration 50, where it had started to learn to play more defensively and tacticly. The trend averaging around the 65 % mark after 50 iterations.

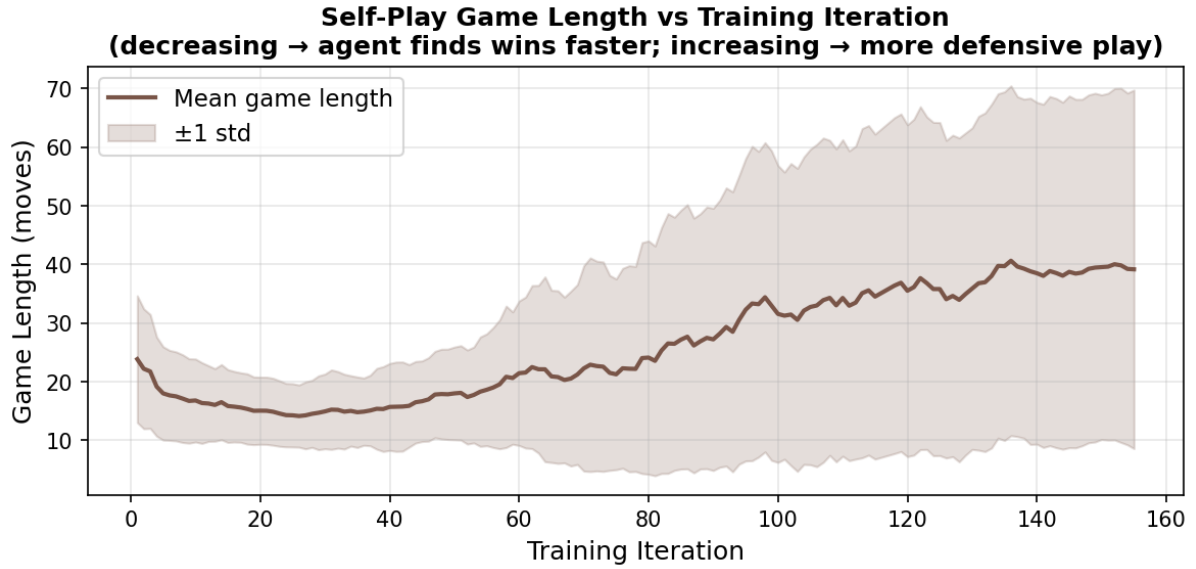

Figure 2: Game length against training iterations

Initially, the mean game length decreased from approximately 28 moves to 15 moves by iteration 40, indicating that the agent learned to efficiently identify and exploit winning sequences rather than engaging in drawn-out positional battles. However, after iteration 40, average game

lengths began to rise, suggesting that self-play successfully focused the agents into stronger defensive capabilities between play themselves. Additionally, the standard deviation of game lengths gradually increased throughout the training process.

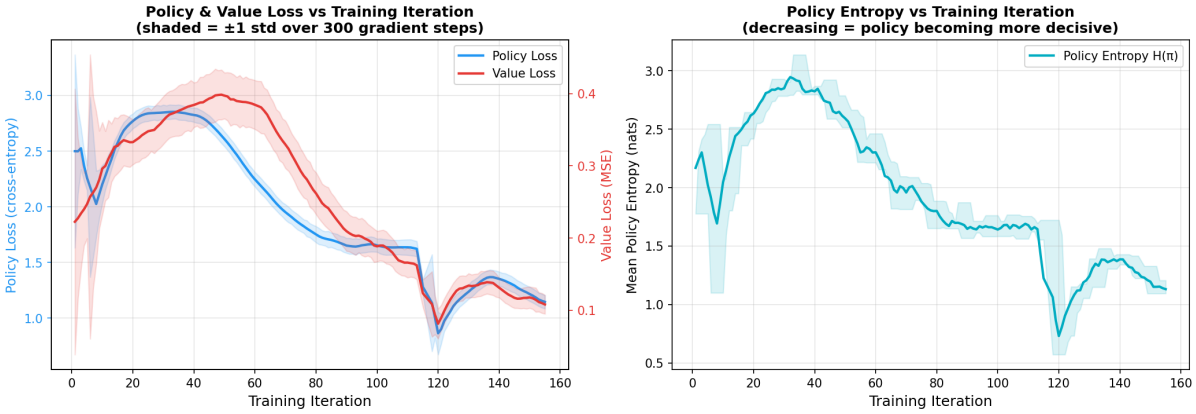

Figure 3: Train Loss Entropy against training iterations

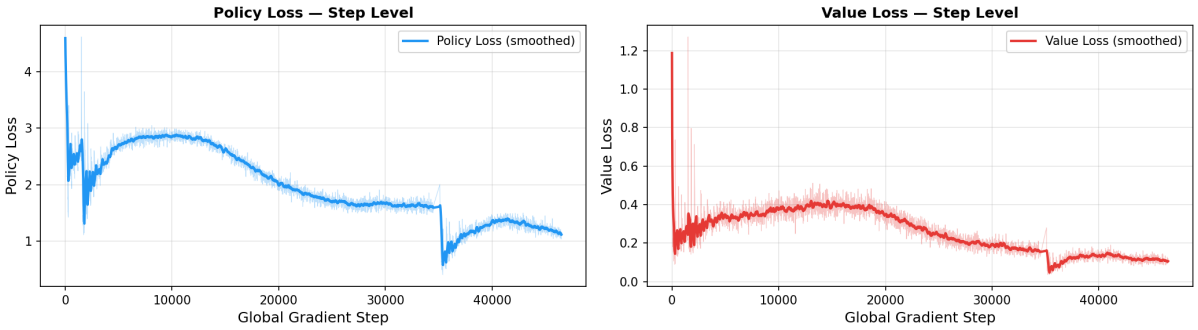

Figure 4: Step Level Losses

At the gradient-step level, policy loss declined sharply at the very start then smoothly from 3 to 1.5 over 40,000 steps confirming stable optimisation throughout.Value loss showed a spike in the first steps as the buffer filled with decisive outcomes, before settling into a gradual decline. There was also a spike at 35,000 due to the stop and starting of the training session, however the consistency between the step-level view and the per-iteration averages validates that the reported iteration metrics faithfully represent the underlying gradient dynamics, with no transient instabilities masked by averaging.

4.4 Ablation Studies

Each version of our training pipeline effectively serves as an ablation, isolating or confounding specific design choices. Below we discuss the effect of each major change.

Network capacity. This was the clearest and most important finding. After training the small network (5 blocks / 64 filters, ∼390k params) extensively under the v2 recipe (77 iterations) and again in a dedicated return-to-baseline run for 100+ iterations, it consistently learned basic blocking and offensive play but remained blind to tactics near the board edges and occasionally missed winning five-in-a-rows. Scaling to the large network (v5, 10 blocks / 128 filters, ∼2M params) while keeping the *same* training recipe produced the strongest agent by far. This confirms that representational capacity — not the training procedure — was the primary bottleneck for 9×9 Gomoku.

Data augmentation. Dihedral augmentation (8× data expansion via rotations and reflections) was introduced in v2 and retained in all subsequent versions. With 50 self-play games per iteration and an average game length of ∼35 moves, augmentation yields approximately $50 \times 35 \times 8 = 14{,}000$ training examples per iteration rather than ∼1,750. This massively accelerates early learning by exposing the network to every tactical pattern in all orientations at no additional self-play cost.

Tree reuse. Tree reuse was introduced in v2. After each move, the MCTS root is advanced to the child corresponding to the played action, preserving all accumulated visit counts and value estimates. Without reuse, each move starts MCTS from scratch; with reuse, move $k{+}1$ benefits from all search statistics accumulated during move k, compounding over the game and producing stronger policy targets at no additional inference cost. This is one of the most impactful modifications in the entire development sequence.

MCTS backup bug fix. The v1 implementation applied the sign flip *after* accumulating the value at each node. In a zero-sum game, the value at a leaf is from the perspective of the player to move, so the flip must occur *before* updating the parent (the opponent's perspective). This bug caused MCTS to systematically favour moves beneficial to the opponent. The fix in v2 was a prerequisite for any meaningful learning.

Learning rate schedule. The aggressive multi-step decay in v2 ($\gamma = 0.1$) collapsed the LR to $\sim 10^{-5}$ by iteration 75, halting all learning. Cosine annealing (v4/v5) provided smooth, sustained decay and produced more stable loss curves without abrupt plateaus. The self-play setting is particularly sensitive to LR scheduling because the training data distribution shifts continuously as the network improves.

Dirichlet noise. The principled choice of Dirichlet α is approximately $10/m$ where m is the average number of legal moves. For 9×9 Gomoku early in a game, $m \approx 80$, giving $\alpha \approx 0.125$. Our default $\alpha = 0.3$ is on the high side; v4 tested $\alpha = 0.15$ (closer to optimal) but its effect was confounded with other simultaneous changes. We retained $\alpha = 0.3$ in v5 as a conservative choice: the higher noise may actually benefit early training by preventing the policy from collapsing to a single opening before the network is strong enough to evaluate alternatives.

4.5 Training Plateau and Warm-Restart Experiments

After 150 iterations of v5 training, the loss plateaued at ∼2.43–2.46 for over 30 iterations. We conducted five systematic attempts to escape this plateau, summarised in Table 5.

Key findings from plateau analysis:

Table 5: Summary of warm-restart attempts to escape the v5 training plateau.

#	Strategy	Base Iter	Peak LR	Buffer	Best Loss / Result
1	High LR, fresh buffer	150	0.008	Fresh 100k	2.31 (iters 151–153) → diverged to 2.63
2a	Conservative LR, old buffer	150	0.002	Original 200k	2.43 — no improvement
2b	Spike-and-settle	150	0.008→0.002	Fresh 100k	Superseded by attempt 3
3	Spike-and-settle from pre-plateau	118	0.008→0.002	Fresh 100k	2.29 (iter 119) → reverted to 2.45
4	Spike-and-settle, seeded buffer	118	0.008→0.002	Seeded 100k	2.47 — old data prevented escape
5	Decoupled warmup, smooth LR	119	0.003	Warmup 76k	2.43 — converged back to plateau

- **Value head converges first**: By mid-training, value loss was ∼0.09 while policy loss was ∼2.36. All further learning is about sharpening the policy distribution over candidate moves.
- **Self-play plateau is self-reinforcing**: Defensive play → draws → defensive training data → more defensive play. The 200k buffer saturated with same-quality drawing games, providing no new learning signal.
- **High-LR spikes find transient basins**: Attempts 1 and 3 achieved loss values of 2.31 and 2.29 respectively, but these improvements were illusory. A small fresh buffer (∼20k examples) is easier to fit due to low diversity. When trained against a full buffer of diverse self-play data (attempt 5: 76k warmup examples), the model converged back to ∼2.43.
- **Feedback loop with sustained high LR**: High LR perturbs the policy → worse self-play data → buffer fills with bad targets → even worse policy → loss climbs continuously (attempt 1 diverged to 2.63).
- **Seeded buffer prevents escape**: When the buffer was pre-loaded with existing data (attempt 4), the old examples anchored the model to the plateau, preventing the spike from having any effect. Compare: fresh-buffer spike reached 2.29 vs. seeded-buffer spike at 2.47.
- **The plateau is likely a capacity limit**: Policy loss ∼2.35 corresponds to spreading probability across ∼10 plausible moves per position. This may represent the architectural ceiling of the 2M-parameter network on 9×9 Gomoku, rather than a training issue.

5 Pure Self-Play vs Mixed-Opponent Training

To investigate how the source of training data affects agent development, our team implemented two distinct training paradigms within the same AlphaZero framework. Both use MCTS guided by a policy-value network, but differ fundamentally in *where the training signal comes from.*

- **Pure self-play**: The agent trains exclusively by playing against itself. All training data is generated from the current network's own policy. This is the approach used in our v1–v5 development line (Section 4) and matches the original AlphaZero paper most closely.
- **Mixed-opponent training**: The agent trains on a mixture of self-play games and games against non-learning opponents. In our implementation, 30% of training games are played against a rule-based heuristic opponent (Section 3), while the remaining 70% are standard self-play. This exposes the network to tactical patterns — forced wins, urgent blocks, threat chains — that rarely emerge in early pure self-play when both sides are weak.

Table 6 compares the two training pipelines.

Table 6: Design comparison between the pure self-play and mixed-opponent training paradigms.

Dimension	**Pure Self-Play**	**Mixed-Opponent Training**
Training opponents	Self only (100%)	Self (70%) + heuristic (30%)
Res. blocks / Filters	10 / 128 (∼2M params)	4 / 64 (∼200k params)
Value head FC	256 hidden units	64 hidden units
Colour plane (plane 2)	Encodes Black/White identity	Constant all-ones (bias only)
MCTS sims (training)	400	200
Self-play style	Batched (50 games, GPU)	Sequential (one game at a time)
Optimizer	SGD (lr=0.01, momentum=0.9)	Adam (lr=0.002)
LR schedule	Cosine ($\eta_{\min} = 5 \times 10^{-4}$)	Cosine ($\eta_{\min} = 1 \times 10^{-4}$)
Value loss weight λ_v	1.0	2.0 (upweighted)
Gradient clipping	None	`clip_grad_norm_` at 1.0
Weight initialisation	PyTorch defaults	Explicit Kaiming (conv) + Xavier (linear)
Evaluation during training	None	Every 5 iters vs. random and heuristic

5.1 Trade-offs Between Paradigms

Early tactical competence. The most significant difference is the mixed-opponent approach's use of heuristic games as a training signal. In early training, a pure self-play agent plays essentially random games against itself; both sides are weak, so sharp tactical situations (an open four that must be blocked, a forced winning sequence) are rare. The network receives few clear examples of "miss this and you lose instantly." Mixed-opponent training solves this bootstrapping problem: the heuristic opponent always blocks four-in-a-rows and always plays winning moves, generating a stream of unambiguous tactical training examples from the very first iteration.

Representational capacity vs. signal quality. The pure self-play pipeline compensates for weaker early training signal with a much larger network (∼2M vs. ∼200k parameters) and higher MCTS simulation count (400 vs. 200). This gives the agent greater representational capacity and stronger per-move search, but requires more iterations to converge. The mixed-opponent pipeline uses a smaller, faster network that converges more quickly in part because it receives higher-quality early training signal from the heuristic games.

State representation. The pure self-play pipeline encodes the current player's colour explicitly (plane 2 is all-ones for Black, all-zeros for White). This is important in Gomoku because Black has a structural first-mover advantage; the network can learn asymmetric strategies (aggressive for Black, reactive for White). The mixed-opponent pipeline uses a constant all-ones bias plane instead, which does not convey colour information — the network must implicitly infer whose turn it is from the stone distribution alone, a weaker signal.

Training stability. The mixed-opponent pipeline includes gradient clipping (max norm 1.0) and upweighted value loss ($\lambda_v = 2.0$), both of which improve training stability. Gradient clipping prevents large spikes from destabilising the shared backbone, while value upweighting ensures the value head converges early, improving MCTS leaf evaluations. Our pure self-play v5 loss breakdown showed the value head converging to ∼0.09 well before the policy loss plateaued (∼2.36), suggesting the value head may over-converge relative to the policy head when weighted equally — upweighting could redistribute gradient signal more productively.

GPU utilisation. The pure self-play pipeline runs all 50 games simultaneously with batched GPU inference, maximising hardware utilisation. The mixed-opponent pipeline runs games sequentially, under-utilising the GPU but simplifying the codebase.

5.2 Synthesis

Neither paradigm is strictly superior. Pure self-play is more faithful to the original AlphaZero philosophy and avoids introducing bias from a hand-coded opponent, but suffers from a cold-start problem where the agent must discover basic tactics from scratch. Mixed-opponent training bootstraps tactical awareness quickly but introduces a dependency on the quality and style of the heuristic opponent — the agent may over-fit to exploiting heuristic-specific weaknesses rather than learning general strategy.

An ideal implementation would combine the strengths of both: the pure self-play pipeline's batched GPU inference, larger network, colour plane encoding, and higher simulation count with the mixed-opponent pipeline's heuristic game mixing for early training, value loss upweighting, and gradient clipping. Concretely, one could use mixed-opponent training for the first N iterations to bootstrap tactical competence, then transition to pure self-play for the remainder of training to develop deeper strategic understanding without heuristic bias.

6 Agent Intelligence and Discussion

6.1 Strategic Capabilities

Against the rule-based heuristic opponent, the v5 agent demonstrates several markers of genuine strategic understanding beyond reactive play:

- **Blocking**: The agent reliably identifies and blocks opponent four-in-a-rows and open-ended three-in-a-rows. The small network (v2) also learned this, but the large network (v5) does so more consistently, including near board edges where the small network had blind spots.
- **Offensive construction**: The agent builds multi-directional threats, placing stones that simultaneously extend two or more partial lines. This forces the opponent into a position where blocking one threat leaves another open — the classical "fork" strategy in Gomoku.
- **Defensive discipline**: The 62% draw rate in the v5 iter-118 vs. iter-143 evaluation demonstrates that the agent learned non-trivial defensive play. Both checkpoints consistently neutralised each other's attacks, indicating the model had internalised threat detection beyond simple pattern matching.

6.2 Limitations

Despite strong tactical play, the agent exhibits clear limitations:

- **Edge blindness (small network)**: The ∼390k-parameter network (v1–v3) consistently missed five-in-a-row threats near the extreme edges of the board. The 3×3 convolution receptive field at a board corner covers only a small fraction of the relevant neighbourhood, and the shallow 5-block network lacked the depth to propagate edge information to the decision layer. The larger v5 network (10 blocks, 128 filters) largely resolved this.
- **Offensive stagnation**: Once both self-play agents learned strong defence, training entered a self-reinforcing plateau: defensive draws dominated self-play data, which trained a more defensive agent, which drew more. The agent learned *not to lose* before it learned *how to win* against an equally-skilled opponent.
- **Policy uncertainty**: Even at the training plateau, the policy head distributed probability across ∼10 candidate moves per position (policy loss ∼2.35). While some of this reflects genuine ambiguity in Gomoku positions, it also suggests the network struggled to commit to a single best move in complex middlegame positions.

6.3 Summary of Lessons Learned

1. **Change one variable at a time**: Versions 3 and 4 each introduced multiple changes simultaneously, making it impossible to diagnose failures. The most productive experiments (v2 baseline, then v5 scale-up) each changed exactly one major variable from the previous working configuration.

2. **Network capacity is the bottleneck**: The small network reached a hard ceiling regardless of training recipe refinements. Scaling to 10 blocks / 128 filters while keeping the proven recipe produced the strongest agent.

3. **Self-play plateaus are structural, not just training artefacts**: Five distinct warm-restart strategies failed to sustainably improve on the plateau loss. The $\sim$2.43 plateau likely represents the architectural limit of our 2M-parameter network on 9×9 Gomoku.

4. **Value head convergence outpaces policy head**: The value loss became negligible ($\sim$0.09) while the policy head still had substantial loss ($\sim$2.36). Future work should investigate whether upweighting the value loss (as in the alternative implementation) or architectural changes to the policy head could shift this balance.

5. **LR scheduling matters more in self-play**: Unlike supervised learning with a fixed dataset, the self-play data distribution shifts as the network improves. Aggressive LR decay kills learning prematurely; sustained high LR destabilises the data generation pipeline. Cosine annealing provided the best trade-off.

7 Team Contributions

Algorithm Lead: text
Environment Engineer: text
Data Analyst: text
Project Manager: text